



ULUSLARARASI KIBRIS ÜNİVERSİTESİ
CYPRUS INTERNATIONAL UNIVERSITY

Project Title: Parking Slot Reservation System

SWEN202 - Software Requirements Analysis and Modelling

Submitted To: Asst. Prof. Dr. Felix Babalola

Group Number: 26

Submitted By:

SAMUEL AKPOGHENE OTOBO – 22100896

GitHub Url: <https://github.com/samuelotobo/Software-Engineering-Project-.git>

Date of Submission:

Table of Contents

Table of Contents	2
Chapter 1: Introduction	4
1.1. Project Overview & Problem Statement	4
1.2. Project Scope & Objectives	5
1.3. Report Structure	5
Chapter 2: Project Management.....	7
2.1 Team Members & Roles	7
2.2 Work Breakdown & Individual Contributions.....	7
2.3 Chosen Software Process Model & Justification	8
2.4 Project Timeline.....	8
Chapter 3: Stakeholder & Requirements Elicitation.....	9
3.1 Stakeholder Identification	9
3.2 Stakeholder Analysis	10
3.3 Requirements Elicitation Techniques Used	10
Chapter 4: Requirements Specification.....	12
4.1 Functional Requirements (FRs)	12
4.2 Non-Functional Requirements (NFRs)	13
Chapter 5: System Modelling	15
5.1 Use Case Model	15
5.1.1 Use Case Diagram.....	15
5.1.2 Detailed Use Case Descriptions:	16
5.2 Dynamic Models	18
5.2.1 Activity Diagrams	18
5.2.2 Sequence Diagrams.....	18
5.3 Structural Model	19
5.3.1 Class Diagram.....	19
5.4 Data Flow Model	21

5.4.1 Context Diagram	21
5.4.2 Level-1 DFD	21
Chapter 6: Implementation	23
6.1 Technology stack details	23
6.2 Key algorithms or code snippets	24
6.3 User interfaces	31
6.4 Open-source code	36
Chapter 7: Testing	37
7.1 Unit testing approach	37
7.2 Test Cases	38
7.3 Acceptance Testing	39
Chapter 8: Conclusion	40
8.1 Project Summary	40
8.2 Challenges Faced & Lessons Learned	40
References	42
Appendices	42

Chapter 1: Introduction

The Parking Slot Reservation System is a web-based software application developed to improve the management and reservation of parking spaces. The system allows users to search for available parking slots, reserve spaces online, cancel reservations, and manage parking activities efficiently. Administrators can monitor parking usage, update slot information, and generate reports through an admin dashboard.

The project applies software engineering principles such as requirements analysis, system modeling, implementation, testing, and documentation to solve a real-world parking management problem.

1.1. Project Overview & Problem Statement

Finding parking spaces in busy areas such as universities, shopping centers, hospitals, and office buildings is often difficult and time-consuming. Drivers usually spend unnecessary time searching for available parking slots manually, which leads to traffic congestion, fuel wastage, delays, and frustration.

Traditional parking systems also make it difficult for administrators to monitor parking slot availability and manage reservations efficiently. In many cases, there is no centralized system that allows users to reserve parking spaces in advance or receive real-time updates about slot availability.

The Parking Slot Reservation System was developed to solve these problems by providing a digital platform where users can:

- Search for available parking slots
- Reserve parking spaces online
- View reservation history
- Cancel reservations when necessary

The system also helps administrators manage parking slots more effectively through real-time monitoring and reservation management tools.

1.2. Project Scope & Objectives

Project Scope

The Parking Slot Reservation System focuses on providing an online parking reservation and management platform for users and administrators.

The System WILL:

- Allow users to register and log into the system
- Display available parking slots in real time
- Allow users to reserve parking spaces
- Allow users to cancel reservations
- Allow administrators to manage parking slots
- Generate parking reservation reports
- Prevent double-booking of parking slots
- Store reservation data in a database

The System WILL NOT:

- Control physical parking barriers or gate systems
- Provide real-time GPS tracking
- Support online payment processing in the current version
- Integrate with external government parking databases
- Include AI-based parking prediction features

Project Objectives

The main objectives of the project are:

1. To reduce the time drivers spend searching for parking spaces.
2. To improve parking slot management efficiency.
3. To provide real-time parking slot availability information.
4. To simplify the parking reservation process.
5. To prevent parking conflicts and double bookings.
6. To provide administrators with monitoring and reporting tools.
7. To develop a responsive and user-friendly web application.

1.3. Report Structure

This report is organized into several chapters, each focusing on a specific part of the project development process.

Chapter 1: Introduction

Provides an overview of the project, problem statement, project scope, objectives, and report structure.

Chapter 2: Project Management

Describes team roles, task distribution, software process model, and project timeline.

Chapter 3: Stakeholder & Requirements Elicitation

Identifies stakeholders and explains the techniques used to gather system requirements.

Chapter 4: Requirements Specification

Presents the functional and non-functional requirements of the system.

Chapter 5: System Modelling

Includes UML diagrams such as use case diagrams, activity diagrams, sequence diagrams, class diagrams, and data flow diagrams.

Chapter 6: Implementation

Explains the technologies used, key algorithms, database integration, and user interface design.

Chapter 7: Testing

Describes the testing methods, test cases, and acceptance testing results.

Chapter 8: Conclusion

Summarizes the project outcomes, challenges faced, lessons learned, and future improvements.

Chapter 2: Project Management

2.1 Team Members & Roles

Team Member	Primary Role
Member 1	Requirements Analysis & Documentation
Member 1	Use Case Modelling & UML Diagrams
Member 1	Frontend Design & User Interface Development
Member 1	Backend Development & Database Integration
Member 1	Testing, Debugging & Report Compilation

2.2 Work Breakdown & Individual Contributions

Task	Description	Primary Member(s)
Stakeholder Analysis	Identified and analyzed system stakeholders	Member 1
Requirements Gathering	Collected and documented system requirements	Member 1
Use Case Modelling	Created use case diagrams and descriptions	Member 1
Activity & Sequence Diagrams	Developed dynamic UML diagrams	Member 1
Class Diagram	Designed system structure and relationships	Member 1
Frontend Development	Designed and implemented user interface	Member 1
Backend Development	Developed APIs and server-side logic	Member 1
Database Design	Designed reservation and slot database structure	Member 1
Testing	Conducted unit and acceptance testing	Member 1
Documentation	Final report formatting and editing	Member 1

2.3 Chosen Software Process Model & Justification

The Agile software development model was selected for this project.

Justification for Choosing Agile

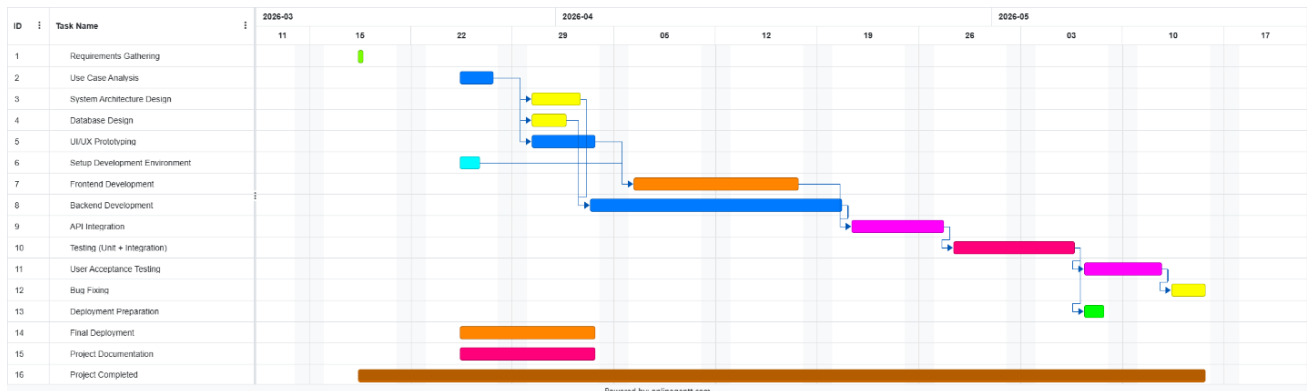
The Agile model was considered suitable because the project required continuous improvements, frequent testing, and flexibility during development. Agile allowed the team to divide the project into smaller tasks and develop features incrementally.

Benefits of Using Agile in This Project

- Easy adaptation to requirement changes
- Faster feedback during development
- Continuous testing and improvement
- Better collaboration among team members
- Faster detection and correction of errors

The project was developed in multiple iterations, where each iteration focused on a specific module such as authentication, parking reservation, admin management, and testing. This approach helped ensure that the final system met user requirements effectively.

2.4 Project Timeline



Chapter 3: Stakeholder & Requirements Elicitation

3.1 Stakeholder Identification

Stakeholders are individuals or groups who interact with the system directly or are affected by its operation. The stakeholders for the Parking Slot Reservation System are categorized into primary and secondary stakeholders.

Primary Stakeholders

1. Drivers / Users

Drivers are the main users of the system. They use the application to search for available parking slots, reserve spaces, cancel reservations, and view booking history. Their main goal is to save time and avoid difficulties in finding parking spaces manually.

2. Parking Administrators

Parking administrators are responsible for managing the parking system. They monitor parking slot availability, update slot statuses, handle reservations, and generate reports related to parking usage.

Secondary Stakeholders

1. System Administrator

The system administrator manages technical aspects of the software such as user permissions, database maintenance, system backups, and security configurations.

2. Facility Management

Facility management personnel use reports generated by the system to analyze parking usage, improve parking efficiency, and support operational decision-making.

3. Security Staff

Security staff monitor parking activities and verify authorized parking reservations. They help ensure that only registered vehicles occupy reserved parking spaces.

3.2 Stakeholder Analysis

Stakeholder	Interest in the System	Influence	Specific Requirements
Drivers / Users	Quick and easy parking reservation	High	Real-time slot availability, simple reservation process, cancellation option
Parking Administrators	Efficient parking management	High	Slot management tools, reservation monitoring, reporting system
System Administrator	Secure and stable operation	Medium	User management, system security, database backup
Facility Management	Better parking resource utilization	Medium	Parking usage reports and analytics
Security Staff	Verification of parking access	Low	Access to reservation information and vehicle details

The drivers and parking administrators are considered the most important stakeholders because they interact directly with the system on a daily basis. Their requirements strongly influenced the system design, especially the reservation module and admin dashboard. Secondary stakeholders mainly focus on security, maintenance, and operational reporting.

3.3 Requirements Elicitation Techniques Used

Several requirements elicitation techniques were used during the development of the Parking Slot Reservation System to gather accurate and useful system requirements.

1. Brainstorming Sessions

The development team conducted brainstorming meetings to discuss possible system features, user needs, and challenges related to parking management. These sessions helped generate initial ideas for the reservation process and admin functionalities.

2. Observation

The team observed how drivers search for parking spaces manually in public parking areas. This helped identify common problems such as wasted time, traffic congestion, and poor parking organization.

3. Interviews

Informal interviews were conducted with potential users and parking administrators to understand their expectations and difficulties with current parking systems.

Information Gathered from Interviews

- Users wanted a faster reservation process.
- Administrators required better slot monitoring tools.
- Security personnel preferred reservation verification features.

4. User Personas

User personas were created to better understand the needs and behaviors of different users such as drivers, administrators, and security staff. These personas helped guide the design of the user interface and system features.

5. Research and Existing System Analysis

The team analyzed existing parking reservation applications and online parking systems to identify useful features and industry best practices.

Outcome of Requirements Elicitation

The elicitation process helped define:

- Functional requirements
- Non-functional requirements
- User interface expectations
- Security and database requirements
- Reporting and management features

These techniques ensured that the final system addresses real-world parking challenges effectively.

Chapter 4: Requirements Specification

4.1 Functional Requirements (FRs)

FR-001: User Registration

ID: FR-001

Title: User Registration

Description: The system shall allow a new user to create an account by providing a valid full name, email address, phone number, and password.

FR-002: User Login

ID: FR-002

Title: User Login

Description: The system shall allow registered users and administrators to log into the system using valid credentials.

FR-003: View Available Parking Slots

ID: FR-003

Title: View Available Parking Slots

Description: The system shall display all available parking slots in real time, including slot number, zone, and availability status.

FR-004: Search Parking Slots

ID: FR-004

Title: Search Parking Slots

Description: The system shall allow users to search for parking slots based on zone, slot type, or availability.

FR-005: Reserve Parking Slot

ID: FR-005

Title: Reserve Parking Slot

Description: The system shall allow users to reserve an available parking slot for a selected date and time.

FR-006: Cancel Reservation

ID: FR-006

Title: Cancel Reservation

Description: The system shall allow users to cancel an existing reservation before the reservation time expires.

FR-007: Reservation Confirmation

ID: FR-007

Title: Reservation Confirmation

Description: The system shall send a confirmation notification after a successful reservation.

FR-008: View Reservation History

ID: FR-008

Title: View Reservation History

Description: The system shall allow users to view their current and previous parking reservations.

FR-009: Manage Parking Slots

ID: FR-009

Title: Manage Parking Slots

Description: The system shall allow administrators to add, update, remove, and manage parking slot information.

FR-010: Update Slot Status

ID: FR-010

Title: Update Slot Status

Description: The system shall allow administrators to change the status of parking slots to available, reserved, or maintenance.

FR-011: Generate Reports

ID: FR-011

Title: Generate Reports

Description: The system shall allow administrators to generate reports related to parking usage and reservations.

FR-012: Prevent Double Booking

ID: FR-012

Title: Prevent Double Booking

Description: The system shall prevent multiple users from reserving the same parking slot at the same time.

FR-013: User Logout

ID: FR-013

Title: User Logout

Description: The system shall allow users to securely log out of the application.

4.2 Non-Functional Requirements (NFRs)

Non-functional requirements describe the quality attributes and operational standards of the system.

Performance Requirements:

NFR-001: Fast Dashboard Loading

The system shall load the dashboard interface within 2 seconds under normal operating conditions.

NFR-002: Real-Time Slot Updates

The system shall update parking slot availability immediately after a reservation or cancellation.

NFR-003: Concurrent Users

The system shall support multiple users accessing the application simultaneously without major performance degradation.

Security Requirements:**NFR-004: Password Encryption**

All user passwords shall be stored in encrypted format within the database.

NFR-005: Authentication Control

Only authenticated users shall access reservation functionalities and admin features.

NFR-006: Secure Data Access

The system shall restrict unauthorized access to reservation and user information.

Usability Requirements:**NFR-007: User-Friendly Interface**

The interface shall be simple and intuitive for non-technical users.

NFR-008: Easy Learning Curve

Users shall be able to learn basic system functions within 10 minutes.

NFR-009: Responsive Design

The system interface shall adapt properly to desktop, tablet, and mobile screen sizes.

Reliability Requirements:**NFR-010: System Availability**

The system shall remain operational and accessible at least 95% of the time.

NFR-011: Data Consistency

Reservation and parking slot data shall remain accurate and synchronized during all transactions.

Maintainability Requirements:**NFR-012: Modular System Design**

The system shall be developed using modular architecture to simplify maintenance and future upgrades.

NFR-013: Code Documentation

The source code shall include proper comments and documentation for future developers.

Scalability Requirements:**NFR-014: Expandable Architecture**

The system shall support future enhancements such as payment integration and GPS navigation.

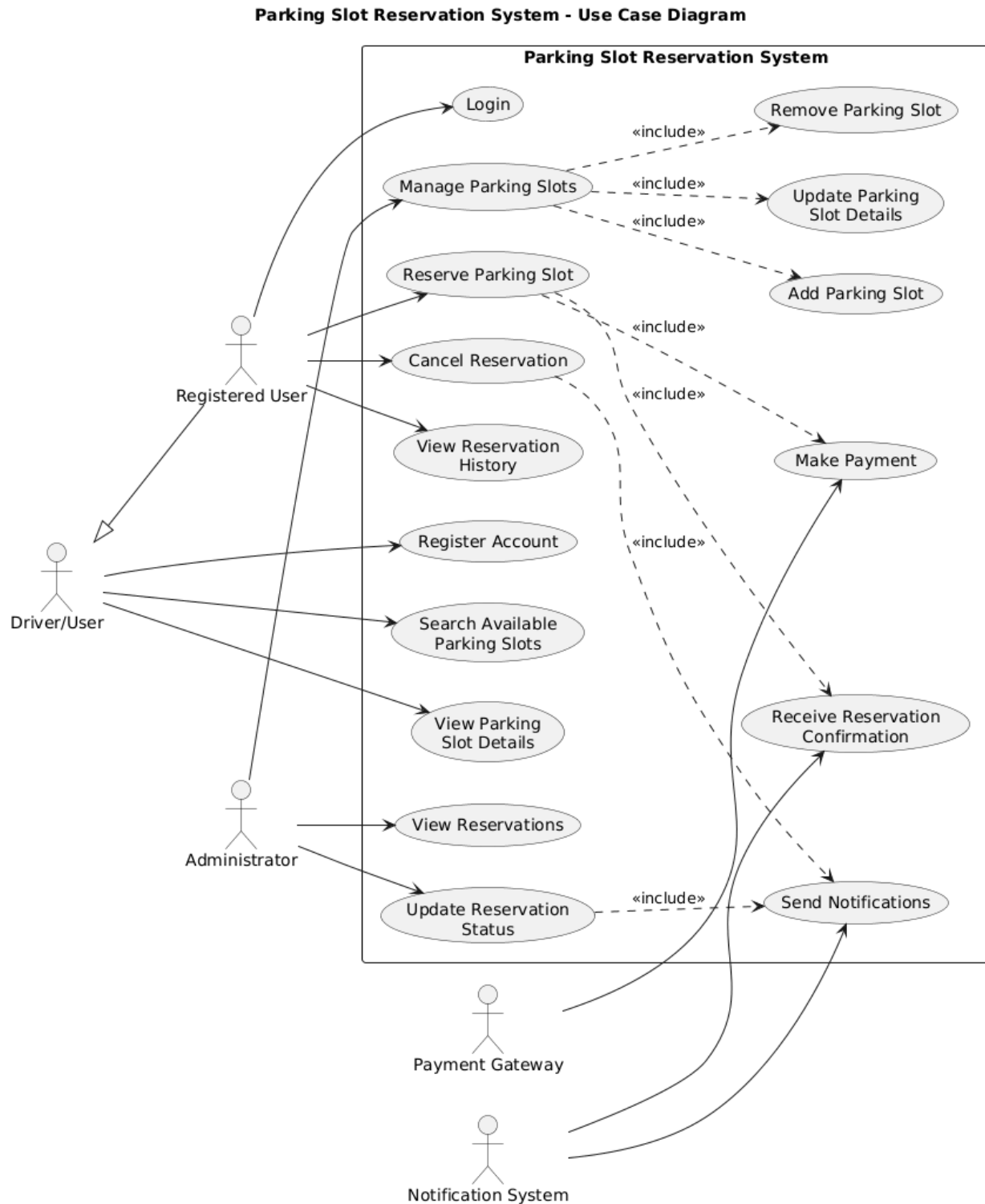
NFR-015: Database Scalability

The database shall support increasing numbers of users, reservations, and parking slots without significant performance loss.

Chapter 5: System Modelling

5.1 Use Case Model

5.1.1 Use Case Diagram



5.1.2 Detailed Use Case Descriptions:

Use Case Name: Register Account

Actor: Driver / User

Pre-condition: The user does not already have an account.

Post-condition: A new user account is created.

Main Success Scenario:

1. The user opens the registration page.
2. The user enters full name, email, phone number, and password.
3. The system validates the information.
4. The system creates the account.
5. The user can now log in.

Alternate Flow:

3a. If the email already exists, the system displays an error message.

Use Case Name: Login

Actor: Registered User / Administrator

Pre-condition: The user must already have a registered account.

Post-condition: The user is authenticated and redirected to the dashboard.

Main Success Scenario:

1. The user opens the login page.
2. The user enters email and password.
3. The system validates the credentials.
4. The system redirects the user to the dashboard.

Alternate Flow:

3a. If credentials are incorrect, the system displays an error message.

Use Case Name: Search Available Parking Slots

Actor: Driver / User

Pre-condition: The system contains parking slot records.

Post-condition: Available parking slots are displayed.

Main Success Scenario:

1. The user opens the parking slot page.
2. The user selects a zone, slot type, or availability filter.
3. The system searches matching parking slots.
4. The system displays available slots.

Alternate Flow:

4a. If no slot is available, the system displays “No available slots found.”

Use Case Name: Reserve Parking Slot

Actor: Registered User

Pre-condition: The user must be logged in and at least one slot must be available.

Post-condition: The selected slot is reserved.

Main Success Scenario:

1. The user searches for available parking slots.
2. The system displays available slots.
3. The user selects a slot.
4. The user enters reservation details.
5. The system checks slot availability.
6. The system saves the reservation.
7. The system updates the slot status to reserved.
8. The system sends confirmation.

Alternate Flow:

5a. If the slot is no longer available, the system asks the user to choose another slot.

Use Case Name: Cancel Reservation

Actor: Registered User

Pre-condition: The user must have an active reservation.

Post-condition: The reservation is cancelled and the slot becomes available.

Main Success Scenario:

1. The user opens reservation history.
2. The user selects the reservation to cancel.
3. The system asks for confirmation.
4. The user confirms cancellation.
5. The system cancels the reservation.
6. The system updates the slot status to available.
7. The system sends cancellation notification.

Alternate Flow:

3a. If cancellation is not allowed, the system displays an error message.

Use Case Name: Manage Parking Slots

Actor: Administrator

Pre-condition: The administrator must be logged in.

Post-condition: Parking slot records are updated.

Main Success Scenario:

1. The administrator opens the admin dashboard.
2. The administrator selects manage parking slots.
3. The administrator adds, updates, removes, or changes slot status.
4. The system validates the changes.
5. The system saves the updated slot information.

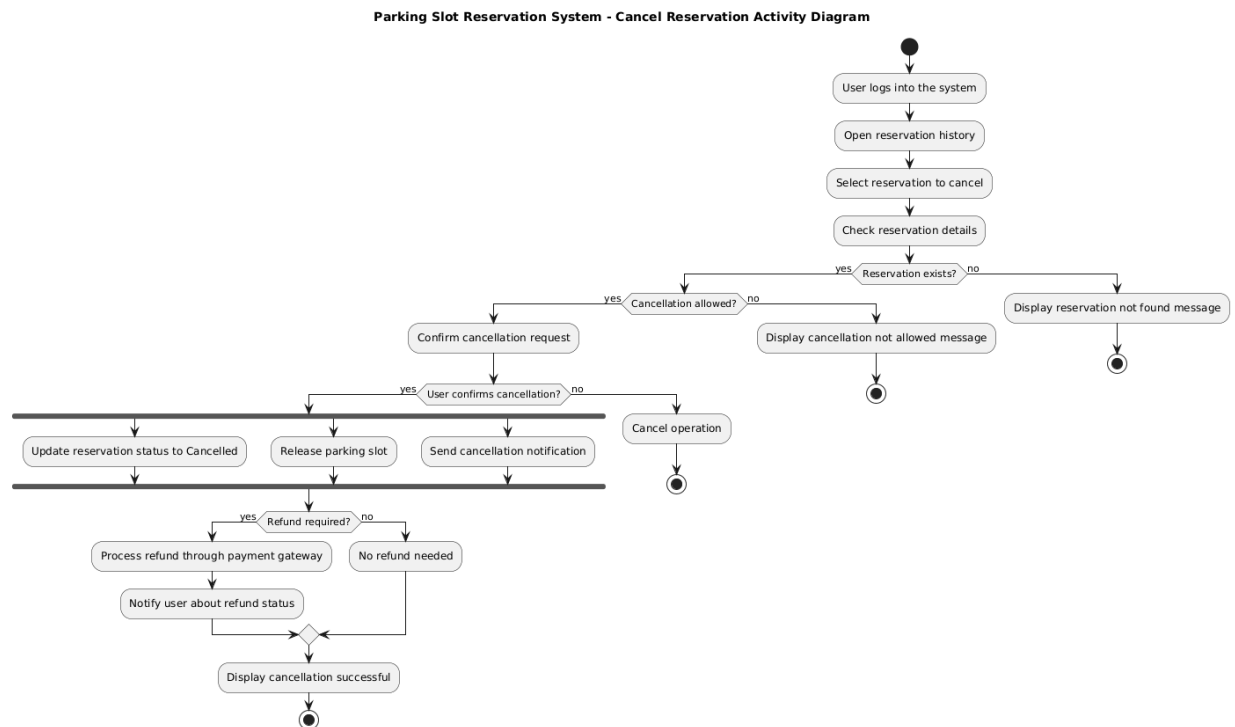
Alternate Flow:

- 4a. If invalid slot details are entered, the system displays an error message.

5.2 Dynamic Models

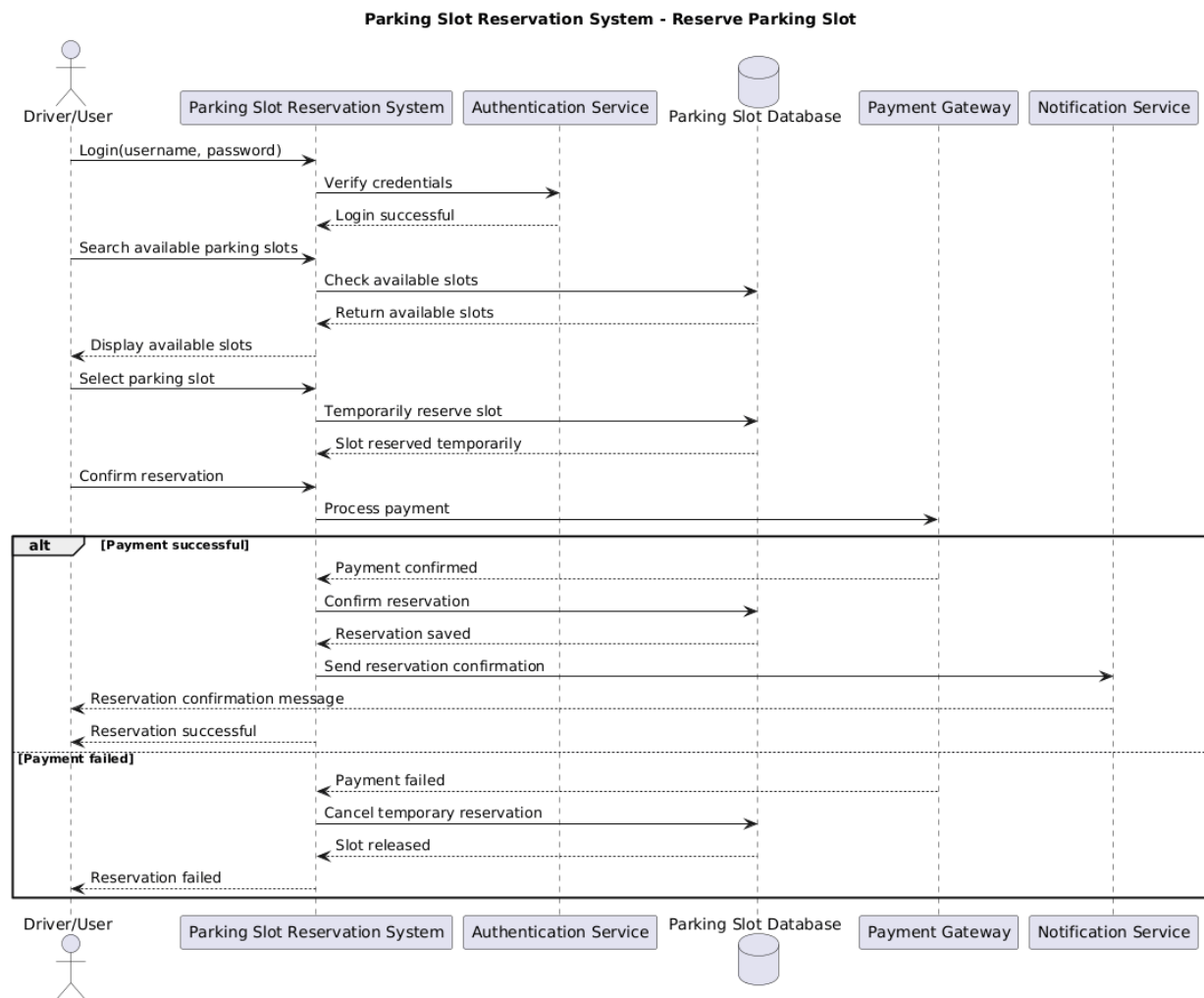
5.2.1 Activity Diagrams

Show the interaction between objects over time for a specific scenario (e.g., the "successful login" scenario from the use case).



5.2.2 Sequence Diagrams

Show the interaction between objects over time for a specific scenario (e.g., the "successful login" scenario from the use case).

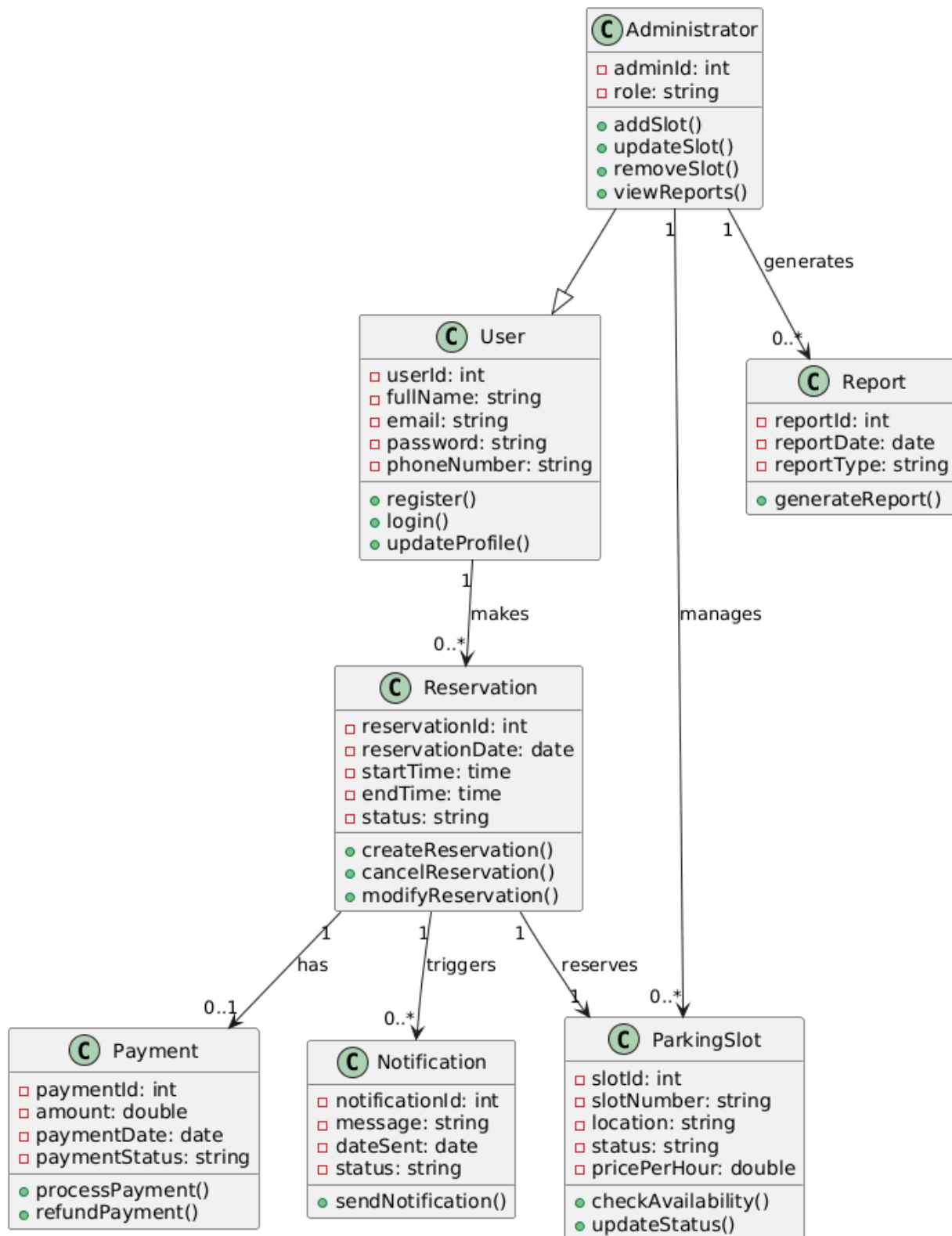


5.3 Structural Model

5.3.1 Class Diagram

Shows the system's classes, their attributes, methods, and relationships (associations, inheritance).

Parking Slot Reservation System - Class Diagram

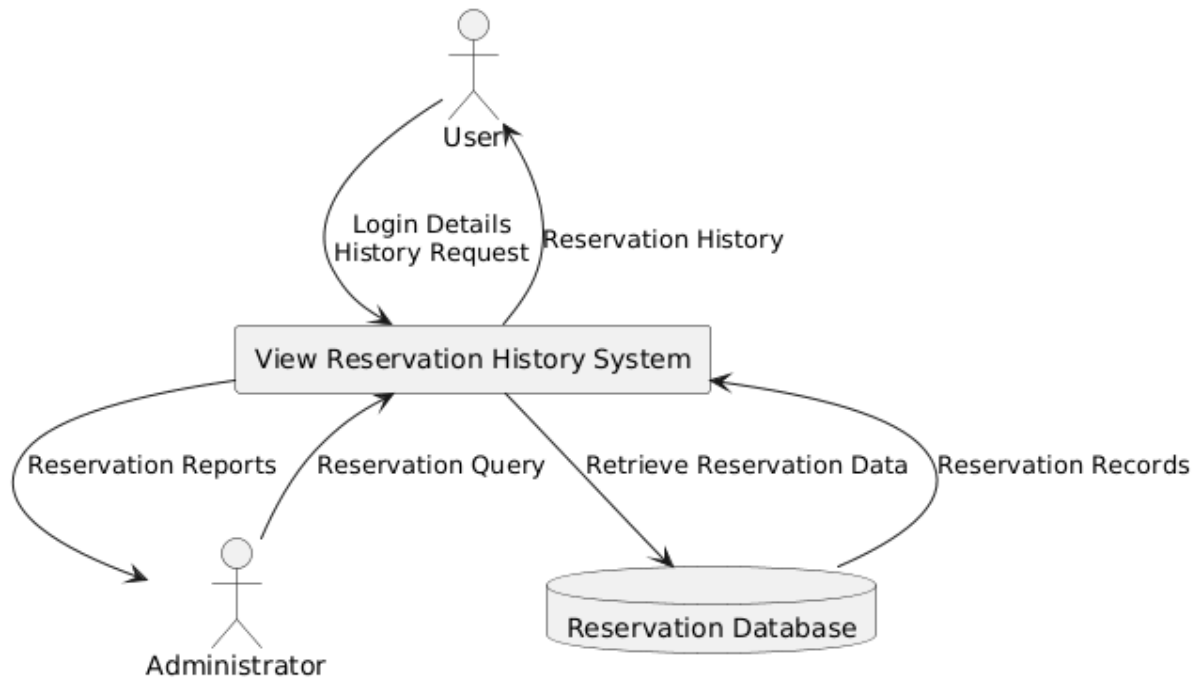


5.4 Data Flow Model

5.4.1 Context Diagram

Shows the system as a single process and its data flows to/from external entities.

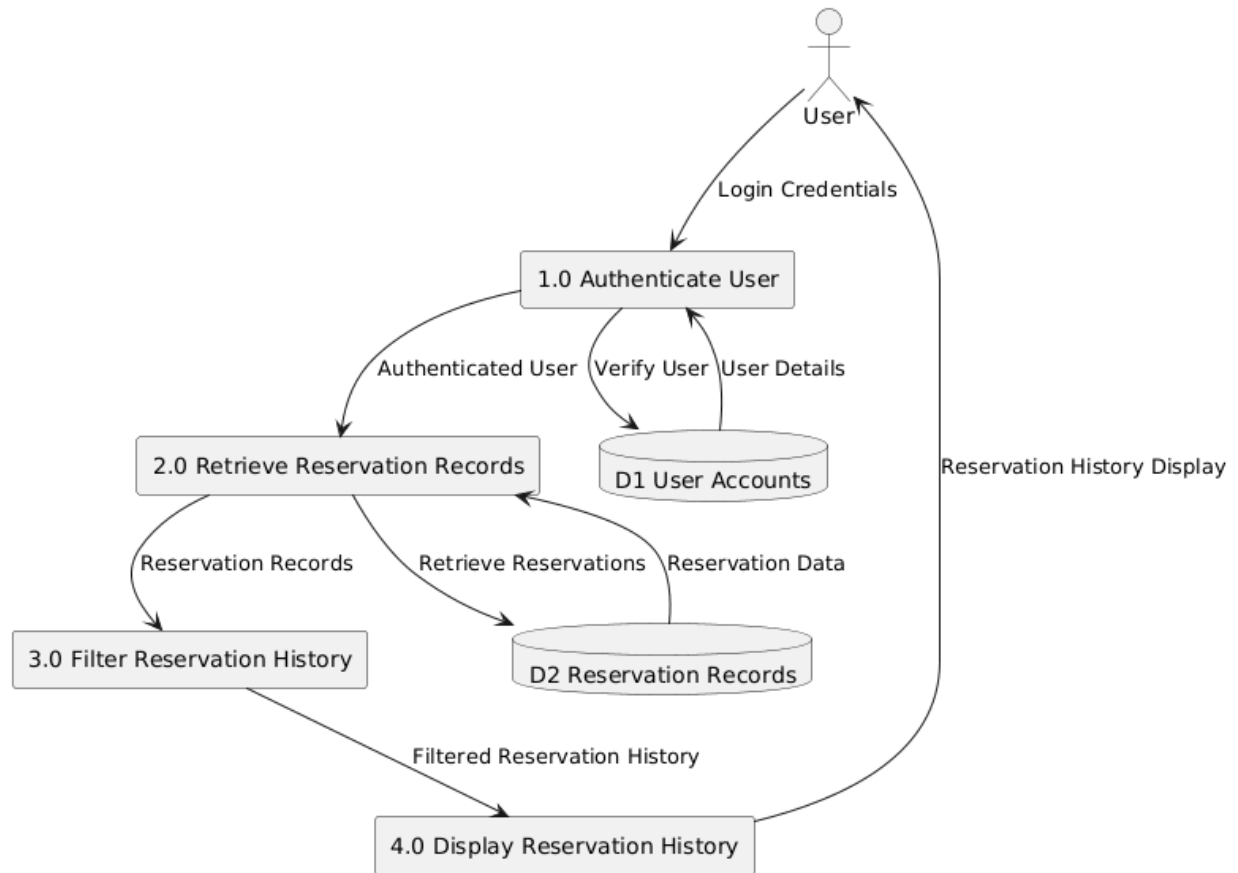
Parking Slot Reservation System - Context Diagram



5.4.2 Level-1 DFD

Decomposes the main process into sub-processes, showing data stores and internal data flows.

Parking Slot Reservation System - Level 1 DFD



Chapter 6: Implementation

6.1 Technology stack details

Layer	Technology Choice	Justification
Frontend (UI)	HTML, CSS, JS	Provides a dynamic user experience with the interface.
Backend (Logic)	Django/Node.js	Team has prior experience
Database	MySQL/PostgreSQL	Suitable for structured database management
Version Control	Git	For collaboration and code backup.
Open Source Libraries	Bootstrap	To speed up development.
Diagrams	Plantuml, Draw.io	Provides tools for complementary diagrams for software and systems design.

6.2 Key algorithms or code snippets

This section describes the main algorithms implemented in the Parking Slot Reservation System and includes code snippets extracted from the developed application.

1. Filtering Available Parking Slots

Purpose: This algorithm allows users to search and filter parking slots based on slot code, zone, and parking type.

Code Snippet

```
const filteredSlots = useMemo(() => {  
  return slots.filter((slot) => {  
    const matchesQuery = [slot.code, slot.zone, slot.type]  
      .join(" ")  
      .toLowerCase()  
      .includes(query.toLowerCase());  
    const matchesZone =  
      zoneFilter === "all"  
      ? true  
      : slot.zone === zoneFilter;  
    return matchesQuery && matchesZone;  
  });  
}, [slots, query, zoneFilter]);
```

Explanation

The algorithm filters parking slots according to the user's search input and selected zone. Only matching slots are displayed on the dashboard, making it easier for users to find suitable parking spaces.

2. Reservation Creation Algorithm

Purpose: This algorithm creates a new reservation and automatically updates the selected parking slot status to Reserved.

Code Snippet

```

const makeReservation = () => {
  const reservation = {
    id: Date.now(),
    slotId: activeSlot.id,
    slotCode: activeSlot.code,
    zone: activeSlot.zone,
    name: form.name,
    vehicle: form.vehicle,
    duration: Number(form.duration),
    date: form.date,
    startTime: form.startTime,
  };
  setReservations((prev) => [reservation, ...prev]);

  setSlots((prev) =>
    prev.map((slot) =>
      slot.id === activeSlot.id
        ? { ...slot, status: "reserved" }
        : slot
    )
  );
};

```

Explanation

When the user confirms a reservation:

- ✓ Reservation details are collected.
- ✓ A reservation object is created.
- ✓ The reservation is added to the reservations list.
- ✓ The selected parking slot status is updated from Available to Reserved.

This prevents other users from booking the same slot.

3. Reservation Cancellation Algorithm

Purpose: This algorithm allows users to cancel a reservation and release the parking slot.

Code Snippet

```
const cancelReservation = (
  reservationId,
  slotId
) => {
  setReservations((prev) =>
    prev.filter(
      (r) => r.id !== reservationId
    )
  );
  setSlots((prev) =>
    prev.map((slot) =>
      slot.id === slotId
        ? {
            ...slot,
            status: "available"
          }
        : slot
    )
  );
};
```

Explanation

When a reservation is cancelled:

- ✓ The reservation is removed from the reservation list.
- ✓ The associated parking slot status is changed back to Available.
- ✓ The slot immediately becomes available for future bookings.

4. Dashboard Statistics Calculation

Purpose: This algorithm generates real-time statistics shown on the dashboard.

Code Snippet

```
const stats = useMemo(() => {  
  const available =  
    slots.filter(  
      (s) => s.status === "available"  
    ).length;  
  const reserved =  
    slots.filter(  
      (s) => s.status === "reserved"  
    ).length;  
  const maintenance =  
    slots.filter(  
      (s) => s.status === "maintenance"  
    ).length;  
  
  return {  
    total: slots.length,  
    available,  
    reserved,  
    maintenance,  
  };  
}, [slots]);
```

Explanation

The algorithm counts:

- ✓ Total parking slots
- ✓ Available slots
- ✓ Reserved slots
- ✓ Maintenance slots

The results are displayed on dashboard summary cards to provide administrators with a real-time overview of parking operations.

5. Add New Parking Slot Algorithm

Purpose: Allows administrators to add new parking slots to the system.

Code Snippet

```
const addSlot = () => {  
  const slot = {  
    id: Date.now(),  
    code: newSlot.code.trim().toUpperCase(),  
    zone: newSlot.zone,  
    type: newSlot.type,  
    price: Number(newSlot.price),  
    status: "available"  
  };  
  
  setSlots((prev) => [...prev, slot]);  
};
```

Explanation

When an administrator enters new slot information:

- ✓ A unique slot ID is generated.
- ✓ Slot details are collected.
- ✓ The slot is added to the system.
- ✓ The slot is automatically marked as available.

This enables administrators to expand parking capacity dynamically.

6. Slot Status Management Algorithm

Purpose: Allows administrators to mark parking spaces as Available, Reserved, or Maintenance.

Code Snippet

```
const setMaintenance = (slotId) => {  
  
  setSlots((prev) =>  
    prev.map((slot) =>  
      slot.id === slotId
```

```
? {  
    ...slot,  
    status: "maintenance"  
}  
: slot  
)  
);  
};
```

Explanation:

This algorithm is used by administrators when a parking slot is temporarily unavailable due to repairs or maintenance. The slot is hidden from normal reservations until it is restored.

Client/App user side:

```
1 const state = {
2   slots: [],
3   reservations: [],
4   stats: null,
5   selectedSlotId: null,
6 };
7
8 const statsEl = document.getElementById('stats');
9 const slotsGridEl = document.getElementById('slotsGrid');
10 const reservationsListEl = document.getElementById('reservationsList');
11 const adminSlotsListEl = document.getElementById('adminSlotsList');
12 const searchInput = document.getElementById('searchInput');
13 const zoneFilter = document.getElementById('zoneFilter');
14 const formMessage = document.getElementById('formMessage');
15 const adminMessage = document.getElementById('adminMessage');
16 const selectedSlotInput = document.getElementById('selectedSlot');
17
18 async function api(path, options = {}) {
19   const res = await fetch(path, {
20     headers: { 'Content-Type': 'application/json' },
21     ...options,
22   });
23   const data = await res.json();
24   if (!res.ok) throw new Error(data.error || 'Request failed');
25   return data;
26 }
27
28 function renderStats() {
29   if (!state.stats) return;
30   const cards = [
31     ['Total Slots', state.stats.total, 'All parking spaces'],
32     ['Available', state.stats.available, 'Ready for booking'],
33     ['Reserved', state.stats.reserved, 'Currently assigned'],
34     ['Maintenance', state.stats.maintenance, 'Temporarily unavailable'],
35   ];
36   stateEl.innerHTML = cards.map(([, title, value, subtitle]) => `
37     <div class="stat-card">
38       <div>
39         </div>
40       </div>
41     `).join('');
42
43   document.querySelectorAll('.slot.available').forEach(button => {
44     button.addEventListener('click', () => {
45       const id = Number(button.dataset.id);
46       const slot = state.slots.find(s => s.id === id);
47       if (!slot) return;
48       state.selectedSlotId = slot.id;
49       selectedSlotInput.value = `${slot.code} - Zone ${slot.zone} (${slot.type})`;
50       formMessage.textContent = `Selected ${slot.code}. Complete the form to reserve it.`;
51     });
52   });
53 }
54
55 function renderReservations() {
56   if (state.reservations.length === 0) {
57     reservationsListEl.innerHTML = `<p class="muted">No active reservations.</p>`;
58     return;
59   }
60   reservationsListEl.innerHTML = state.reservations.map(r => `
61     <div class="list-item">
62       <div class="list-item-info">
63         <div style="display: flex; gap: 8px; align-items: center; flex-wrap: wrap;">
64           <strong>${r.slot.code}</strong>
65           <span class="badge zone">Zone ${r.zone}</span>
66         </div>
67         <span>${r.user.name} - ${r.vehicle.plate}</span>
68         <span class="muted">${r.booking.date} at ${r.start.time} for ${r.duration.hours} hour(s)</span>
69       </div>
70       <button class="btn small danger" onclick="cancelReservation(${r.id})">Cancel</button>
71     </div>
72 `).join('');
73 }
74
75 window.cancelReservation = cancelReservation;
76
77 async function loadData() {
78   const query = encodeURIComponent(searchInput.value.trim());
79   const zone = encodeURIComponent(zoneFilter.value);
80   const [stats, slots, reservations] = await Promise.all([
81     api('/api/stats'),
82     api('/api/slots?q=' + query + '&zone=' + zone),
83     api('/api/reservations'),
84   ]);
85   state.stats = stats;
86   state.slots = slots;
87   state.reservations = reservations;
88   renderStats();
89   renderSlots();
90   renderReservations();
91 }
92
93 async function cancelReservation(id) {
94   try {
95     await api('/api/reservations/' + id, { method: 'DELETE' });
96     formMessage.textContent = 'Reservation cancelled successfully.';
97     await loadData();
98   } catch (error) {
99     formMessage.textContent = error.message;
100   }
101 }
102
103 window.cancelReservation = cancelReservation;
104
105 async function updateSlotStatus(id, status) {
106   try {
107     await api('/api/slots/' + id + '/status', {
108       method: 'PATCH',
109       body: JSON.stringify({ status }),
110     });
111     adminMessage.textContent = `Slot updated to ${status}`;
112     await loadData();
113   } catch (error) {
114     adminMessage.textContent = error.message;
115   }
116 }
117
118 window.updateSlotStatus = updateSlotStatus;
119
120 document.getElementById('reservationForm').addEventListener('submit', async (event) => {
121   event.preventDefault();
122   if (!state.selectedSlotId) {
123     formMessage.textContent = 'Please select an available slot first.';
124     return;
125   }
126   const payload = {
127     user_name: document.getElementById('name').value.trim(),
128     user_email: document.getElementById('email').value.trim(),
129     vehicle_plate: document.getElementById('plate').value.trim(),
130     booking_date: document.getElementById('date').value,
131     start_time: document.getElementById('time').value,
132     duration_hours: Number(document.getElementById('duration').value),
133     slot_id: state.selectedSlotId,
134   };
135   try {
136     await api('/api/reservations', {
137       method: 'POST',
138       body: JSON.stringify(payload),
139     });
140     formMessage.textContent = 'Reservation created successfully.';
141     event.target.reset();
142     selectedSlotInput.value = '';
143     state.selectedSlotId = null;
144     await loadData();
145   } catch (error) {
146     formMessage.textContent = error.message;
147   }
148 });
149
150 document.getElementById('slotForm').addEventListener('submit', async (event) => {
151   event.preventDefault();
152   const payload = {
153     code: document.getElementById('slotCode').value.trim(),
154     zone: document.getElementById('slotZone').value,
155     type: document.getElementById('slotType').value,
156     price_per_hour: Number(document.getElementById('slotPrice').value),
157   };
158   try {
159     await api('/api/slots', {
160       method: 'POST',
161       body: JSON.stringify(payload),
162     });
163     adminMessage.textContent = 'New slot added successfully.';
164     event.target.reset();
165     document.getElementById('slotZone').value = 'A';
166     document.getElementById('slotType').value = 'Compact';
167     await loadData();
168   } catch (error) {
169     adminMessage.textContent = error.message;
170   }
171 });
172
173 searchInput.addEventListener('input', loadData);
174 zoneFilter.addEventListener('change', loadData);
175
176 const now = new Date();
177 document.getElementById('date').value = now.toISOString().slice(0, 10);
178 document.getElementById('time').value = '10:00';
179
180 loadData();
```

```
1 public > # app.js > @ state.slots.map() callback
2 function renderStats() {
3   stateEl.innerHTML = cards.map(([, title, value, subtitle]) => `
4     <div class="stat-card">
5       <div>
6         </div>
7       </div>
8     `).join('');
9 }
10
11 function slotMarkup(slot) {
12   const disabled = slot.status !== 'available' ? 'disabled' : '';
13   return `
14     <button class="slot ${slot.status} ${disabled}" data-id="${slot.id}" ${slot.status !== 'available' ? 'disabled' : ''}>
15       <div class="slot-top"><span>${slot.code}</span><span>${slot.type}</span></div>
16       <small>Zone ${slot.zone}</small>
17       <small>${slot.price_per_hour}/hour</small>
18     </button>
19 `;
20 }
21
22 function renderSlots() {
23   slotsGridEl.innerHTML = state.slots.map(slotMarkup).join('');
24   adminSlotsListEl.innerHTML = state.slots.map(slot => `
25     <div class="list-item">
26       <div class="list-item-info">
27         <div style="display: flex; gap: 8px; align-items: center; flex-wrap: wrap;">
28           <strong>${slot.code}</strong>
29           <span class="badge zone">Zone ${slot.zone}</span>
30           <span class="badge">${slot.status}</span>
31         </div>
32         <span class="muted">${slot.type} • ${slot.price_per_hour}/hour</span>
33       </div>
34       <div style="display: flex; gap: 8px; flex-wrap: wrap;">
35         <button class="btn small outline" onclick="updateSlotStatus(${slot.id}, 'available')>Set Available</button>
36         <button class="btn small secondary" onclick="updateSlotStatus(${slot.id}, 'reserved')>Set Reserved</button>
37         <button class="btn small danger" onclick="updateSlotStatus(${slot.id}, 'maintenance')>Maintenance</button>
38       </div>
39     </div>
40 `).join('');
41 }
42
43 async function loadData() {
44   const query = encodeURIComponent(searchInput.value.trim());
45   const zone = encodeURIComponent(zoneFilter.value);
46   const [stats, slots, reservations] = await Promise.all([
47     api('/api/stats'),
48     api('/api/slots?q=' + query + '&zone=' + zone),
49     api('/api/reservations'),
50   ]);
51   state.stats = stats;
52   state.slots = slots;
53   state.reservations = reservations;
54   renderStats();
55   renderSlots();
56   renderReservations();
57 }
58
59 async function cancelReservation(id) {
60   try {
61     await api('/api/reservations/' + id, { method: 'DELETE' });
62     formMessage.textContent = 'Reservation cancelled successfully.';
63     await loadData();
64   } catch (error) {
65     formMessage.textContent = error.message;
66   }
67 }
68
69 window.cancelReservation = cancelReservation;
70
71 async function updateSlotStatus(id, status) {
72   try {
73     await api('/api/slots/' + id + '/status', {
74       method: 'PATCH',
75       body: JSON.stringify({ status }),
76     });
77     adminMessage.textContent = `Slot updated to ${status}`;
78     await loadData();
79   } catch (error) {
80     adminMessage.textContent = error.message;
81   }
82 }
83
84 window.updateSlotStatus = updateSlotStatus;
85
86 document.getElementById('reservationForm').addEventListener('submit', async (event) => {
87   event.preventDefault();
88   if (!state.selectedSlotId) {
89     formMessage.textContent = 'Please select an available slot first.';
90     return;
91   }
92   const payload = {
93     user_name: document.getElementById('name').value.trim(),
94     user_email: document.getElementById('email').value.trim(),
95     vehicle_plate: document.getElementById('plate').value.trim(),
96     booking_date: document.getElementById('date').value,
97     start_time: document.getElementById('time').value,
98     duration_hours: Number(document.getElementById('duration').value),
99     slot_id: state.selectedSlotId,
100   };
101   try {
102     await api('/api/reservations', {
103       method: 'POST',
104       body: JSON.stringify(payload),
105     });
106     formMessage.textContent = 'Reservation created successfully.';
107     event.target.reset();
108     selectedSlotInput.value = '';
109     state.selectedSlotId = null;
110     await loadData();
111   } catch (error) {
112     formMessage.textContent = error.message;
113   }
114 });
115
116 document.getElementById('slotForm').addEventListener('submit', async (event) => {
117   event.preventDefault();
118   const payload = {
119     code: document.getElementById('slotCode').value.trim(),
120     zone: document.getElementById('slotZone').value,
121     type: document.getElementById('slotType').value,
122     price_per_hour: Number(document.getElementById('slotPrice').value),
123   };
124   try {
125     await api('/api/slots', {
126       method: 'POST',
127       body: JSON.stringify(payload),
128     });
129     adminMessage.textContent = 'New slot added successfully.';
130     event.target.reset();
131     document.getElementById('slotZone').value = 'A';
132     document.getElementById('slotType').value = 'Compact';
133     await loadData();
134   } catch (error) {
135     adminMessage.textContent = error.message;
136   }
137 });
138
139 searchInput.addEventListener('input', loadData);
140 zoneFilter.addEventListener('change', loadData);
141
142 const now = new Date();
143 document.getElementById('date').value = now.toISOString().slice(0, 10);
144 document.getElementById('time').value = '10:00';
145
146 loadData();
```

```
176 document.getElementById('reservationForm').addEventListener('submit', async (event) => {
177   event.preventDefault();
178   if (!state.selectedSlotId) {
179     formMessage.textContent = 'Please select an available slot first.';
180     return;
181   }
182   const payload = {
183     user_name: document.getElementById('name').value.trim(),
184     user_email: document.getElementById('email').value.trim(),
185     vehicle_plate: document.getElementById('plate').value.trim(),
186     booking_date: document.getElementById('date').value,
187     start_time: document.getElementById('time').value,
188     duration_hours: Number(document.getElementById('duration').value),
189     slot_id: state.selectedSlotId,
190   };
191   try {
192     await api('/api/reservations', {
193       method: 'POST',
194       body: JSON.stringify(payload),
195     });
196     formMessage.textContent = 'Reservation created successfully.';
197     event.target.reset();
198     selectedSlotInput.value = '';
199     state.selectedSlotId = null;
200     await loadData();
201   } catch (error) {
202     formMessage.textContent = error.message;
203   }
204 });
205
206 document.getElementById('slotForm').addEventListener('submit', async (event) => {
207   event.preventDefault();
208   const payload = {
209     code: document.getElementById('slotCode').value.trim(),
210     zone: document.getElementById('slotZone').value,
211     type: document.getElementById('slotType').value,
212     price_per_hour: Number(document.getElementById('slotPrice').value),
213   };
214   try {
215     await api('/api/slots', {
216       method: 'POST',
217       body: JSON.stringify(payload),
218     });
219     adminMessage.textContent = 'New slot added successfully.';
220     event.target.reset();
221     document.getElementById('slotZone').value = 'A';
222     document.getElementById('slotType').value = 'Compact';
223     await loadData();
224   } catch (error) {
225     adminMessage.textContent = error.message;
226   }
227 });
228
229 searchInput.addEventListener('input', loadData);
230 zoneFilter.addEventListener('change', loadData);
231
232 const now = new Date();
233 document.getElementById('date').value = now.toISOString().slice(0, 10);
234 document.getElementById('time').value = '10:00';
235
236 loadData();
```

Server Side:

```
1  const express = require('express');
2  const cors = require('cors');
3  const path = require('path');
4  const db = require('./db');
5
6  const app = express();
7  const PORT = process.env.PORT || 3000;
8
9  app.use(cors());
10 app.use(express.json());
11 app.use(express.static(path.join(__dirname, 'public')));
12
13 function stats() {
14   const total = db.prepare('SELECT COUNT(*) AS count FROM slots').get().count;
15   const available = db.prepare('SELECT COUNT(*) AS count FROM slots WHERE status = \'available\'');
16   const reserved = db.prepare('SELECT COUNT(*) AS count FROM slots WHERE status = \'reserved\'');
17   const maintenance = db.prepare('SELECT COUNT(*) AS count FROM slots WHERE status = \'maintena');
18   return { total, available, reserved, maintenance };
19 }
20
21 app.get('/api/health', (_req, res) => {
22   res.json({ ok: true, message: 'Parking API is running' });
23 });
24
25 app.get('/api/stats', (_req, res) => {
26   res.json(stats());
27 });
28
29 app.get('/api/slots', (req, res) => {
30   const zone = req.query.zone;
31   const q = (req.query.q || '').trim().toLowerCase();
32   let rows = db.prepare('SELECT * FROM slots ORDER BY zone, code').all();
33   if (zone && zone !== 'all') rows = rows.filter(r => r.zone === zone);
34   if (q) rows = rows.filter(r => `${r.code} ${r.zone} ${r.type}`.toLowerCase().includes(q));
35   res.json(rows);
36 });
```

```
80
81 app.post('/api/reservations', (req, res) => {
82   const { user_name, user_email, vehicle_plate, slot_id, booking_date, start_time, duration } = req.body;
83   if (!user_name || !user_email || !vehicle_plate || !slot_id || !booking_date || !start_time || !duration)
84     return res.status(400).json({ error: 'All reservation fields are required' });
85
86   const slot = db.prepare('SELECT * FROM slots WHERE id = ?').get(Number(slot_id));
87   if (!slot) return res.status(404).json({ error: 'Slot not found' });
88   if (slot.status !== 'available') {
89     return res.status(400).json({ error: 'This slot is not currently available' });
90   }
91
92   const result = db.prepare(`
93     INSERT INTO reservations (user_name, user_email, vehicle_plate, slot_id, booking_date, start_time, duration)
94     VALUES (?, ?, ?, ?, ?, ?, ?)
95   `).run(user_name, user_email, vehicle_plate, Number(slot_id), booking_date, start_time, duration);
96
97   db.prepare('UPDATE slots SET status = \'reserved\' WHERE id = ?').run(slot_id);
98
99   const reservation = db.prepare(`
100     SELECT r.*, s.code AS slot_code, s.zone AS zone
101     FROM reservations r
102     JOIN slots s ON s.id = r.slot_id
103     WHERE r.id = ?
104   `).get(result.lastInsertRowid);
105
106   res.status(201).json(reservation);
107 });
108
109 app.delete('/api/reservations/:id', (req, res) => {
110   const id = Number(req.params.id);
111   const reservation = db.prepare('SELECT * FROM reservations WHERE id = ? AND status = \'active\'').get(id);
112   if (!reservation) return res.status(404).json({ error: 'Active reservation not found' });
113
114   db.prepare('UPDATE reservations SET status = \'cancelled\' WHERE id = ?').run(id);
115   db.prepare('UPDATE slots SET status = \'available\' WHERE id = ?').run(reservation.slot_id);
116   res.json({ success: true });
117 });
```

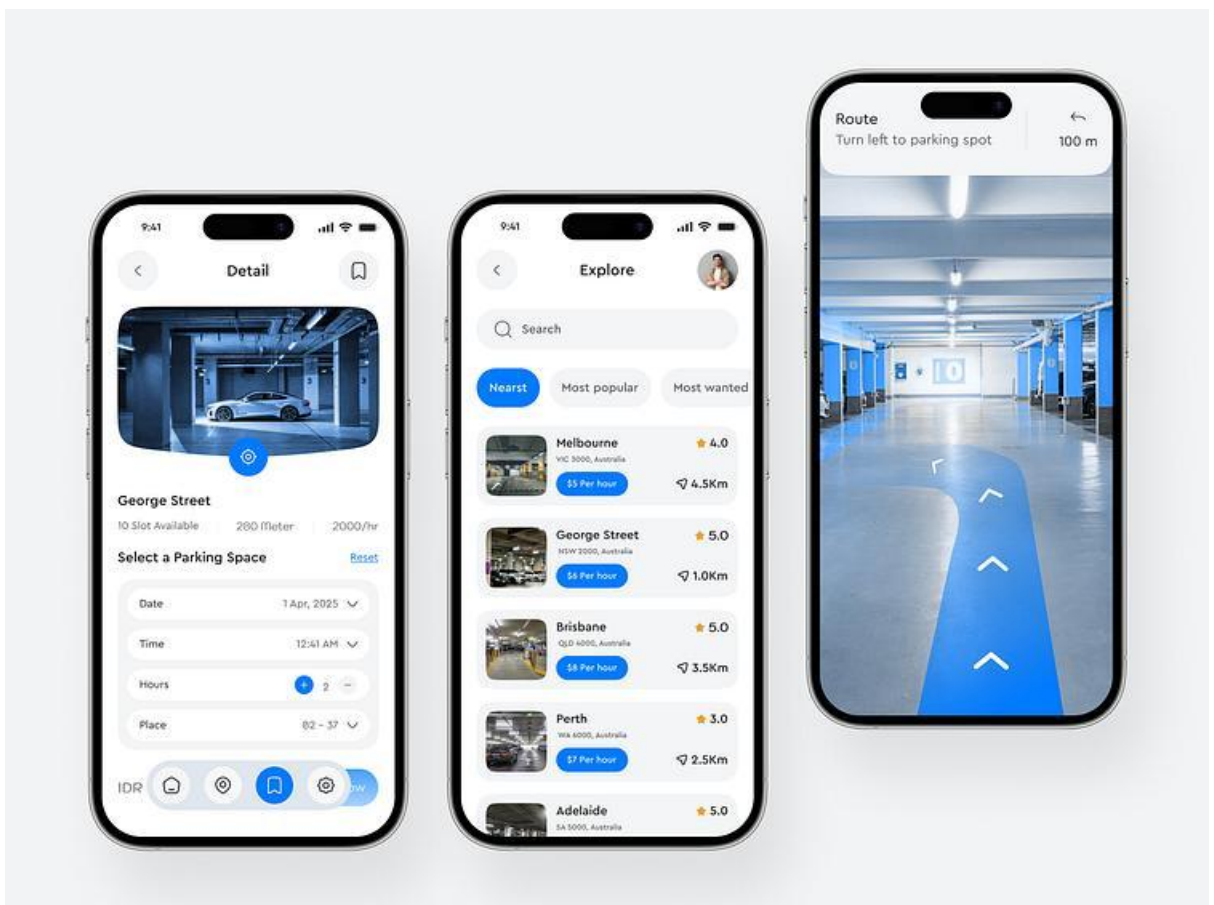
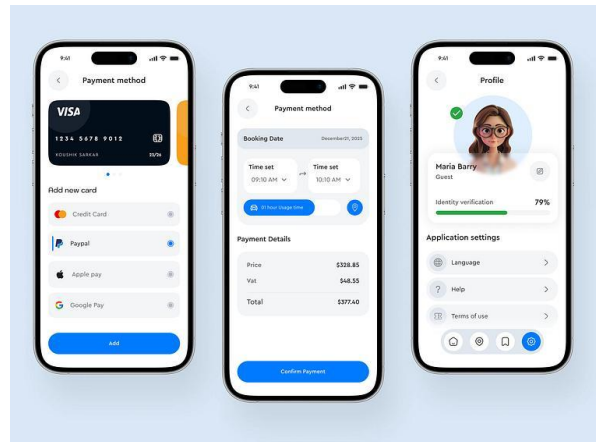
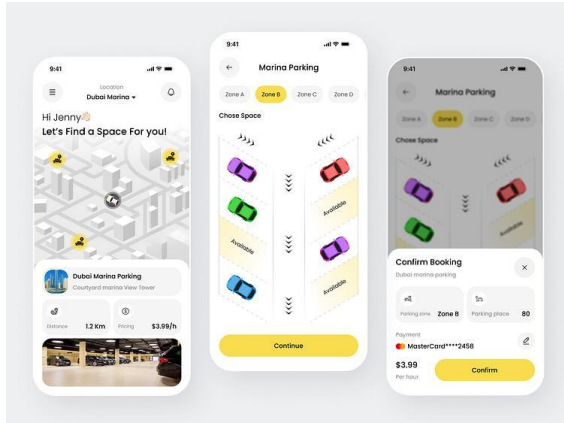
```
38 app.post('/api/slots', (req, res) => {
39   const { code, zone, type, price_per_hour } = req.body;
40   if (!code || !zone || !type || !price_per_hour) {
41     return res.status(400).json({ error: 'code, zone, type, and price_per_hour are required' });
42   }
43   try {
44     const result = db.prepare(`
45       INSERT INTO slots (code, zone, type, price_per_hour, status) VALUES (?, ?, ?, ?, ?)
46     `).run(String(code).toUpperCase(), zone, type, Number(price_per_hour), 'available');
47     const slot = db.prepare('SELECT * FROM slots WHERE id = ?').get(result.lastInsertRowid);
48     res.status(201).json(slot);
49   } catch (error) {
50     return res.status(400).json({ error: 'Slot code must be unique' });
51   }
52 });
53
54 app.patch('/api/slots/:id/status', (req, res) => {
55   const { status } = req.body;
56   const id = Number(req.params.id);
57   if (!['available', 'reserved', 'maintenance'].includes(status)) {
58     return res.status(400).json({ error: 'Invalid status value' });
59   }
60   const slot = db.prepare('SELECT * FROM slots WHERE id = ?').get(id);
61   if (!slot) return res.status(404).json({ error: 'Slot not found' });
62
63   if (status !== 'reserved') {
64     db.prepare('UPDATE reservations SET status = \'cancelled\' WHERE slot_id = ? AND status = \'reserved\'').run(slot.id);
65   }
66   db.prepare('UPDATE slots SET status = ? WHERE id = ?').run(status, id);
67   res.json(db.prepare('SELECT * FROM slots WHERE id = ?').get(id));
68 });
69
70 app.get('/api/reservations', (_req, res) => {
71   const rows = db.prepare(`
72     SELECT r.*, s.code AS slot_code, s.zone AS zone
73     FROM reservations r
74     JOIN slots s ON s.id = r.slot_id
75   `).all();
76   res.json(rows);
77 });
```

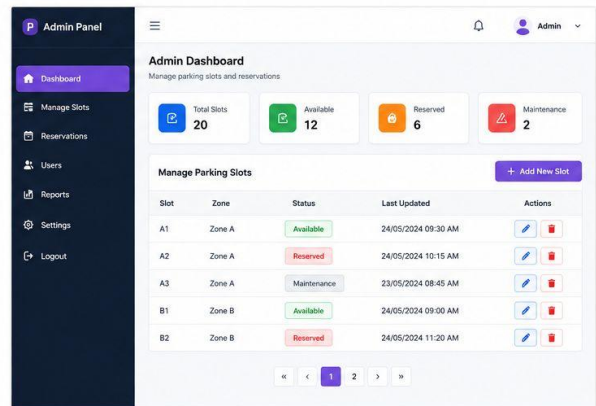
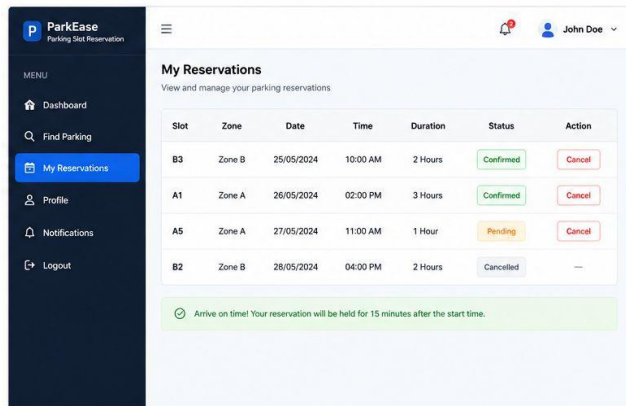
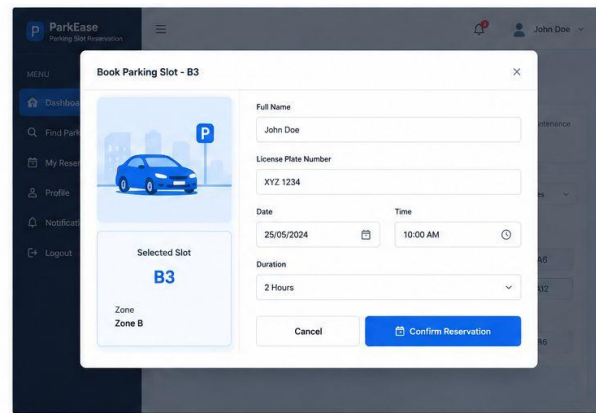
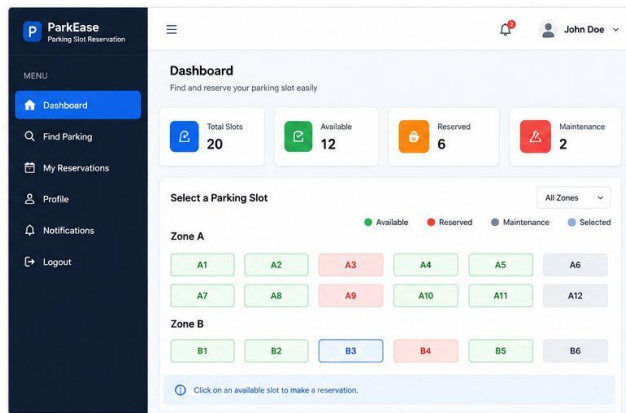
```
81 app.post('/api/reservations', (req, res) => {
82   const reservation = db.prepare(`
83     INSERT INTO reservations (user_name, user_email, vehicle_plate, slot_id, booking_date, start_time, duration)
84     VALUES (?, ?, ?, ?, ?, ?, ?)
85   `).run(user_name, user_email, vehicle_plate, Number(slot_id), booking_date, start_time, duration);
86   res.status(201).json(reservation);
87 });
88
89 app.delete('/api/reservations/:id', (req, res) => {
90   const id = Number(req.params.id);
91   const reservation = db.prepare('SELECT * FROM reservations WHERE id = ? AND status = \'active\'').get(id);
92   if (!reservation) return res.status(404).json({ error: 'Active reservation not found' });
93
94   db.prepare('UPDATE reservations SET status = \'cancelled\' WHERE id = ?').run(id);
95   db.prepare('UPDATE slots SET status = \'available\' WHERE id = ?').run(reservation.slot_id);
96   res.json({ success: true });
97 });
98
99 app.get('*', (_req, res) => {
100   res.sendFile(path.join(__dirname, 'public', 'index.html'));
101 });
102
103 app.listen(PORT, () => {
104   console.log(`Server running on http://localhost:${PORT}`);
105 });
```

Database:

```
dbjs X
dbjs > ...
1 const Database = require('better-sqlite3');
2 const db = new Database('parking.db');
3
4 db.pragma('journal_mode = WAL');
5
6 db.exec(`
7 CREATE TABLE IF NOT EXISTS users (
8   id INTEGER PRIMARY KEY AUTOINCREMENT,
9   name TEXT NOT NULL,
10  email TEXT UNIQUE NOT NULL,
11  role TEXT NOT NULL DEFAULT 'user'
12 );
13
14 CREATE TABLE IF NOT EXISTS slots (
15   id INTEGER PRIMARY KEY AUTOINCREMENT,
16   code TEXT UNIQUE NOT NULL,
17   zone TEXT NOT NULL,
18   type TEXT NOT NULL,
19   price_per_hour REAL NOT NULL,
20   status TEXT NOT NULL CHECK(status IN ('available','reserved','maintenance')) DEFAULT 'available'
21 );
22
23 CREATE TABLE IF NOT EXISTS reservations (
24   id INTEGER PRIMARY KEY AUTOINCREMENT,
25   user_name TEXT NOT NULL,
26   user_email TEXT NOT NULL,
27   vehicle_plate TEXT NOT NULL,
28   slot_id INTEGER NOT NULL,
29   booking_date TEXT NOT NULL,
30   start_time TEXT NOT NULL,
31   duration_hours INTEGER NOT NULL,
32   status TEXT NOT NULL CHECK(status IN ('active','cancelled')) DEFAULT 'active',
33   created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
34   FOREIGN KEY(slot_id) REFERENCES slots(id)
35 );
36 `);
37
38 const slotCount = db.prepare('SELECT COUNT(*) AS count FROM slots').get().count;
39 if (slotCount === 0) {
40   const insert = db.prepare('INSERT INTO slots (code, zone, type, price_per_hour, status) VALUES (?, ?, ?, ?, ?)');
41   const slots = [
42     ['A-01', 'A', 'Compact', 5, 'available'],
43     ['A-02', 'A', 'SUV', 6, 'available'],
44     ['A-03', 'A', 'Compact', 5, 'reserved'],
45     ['B-01', 'B', 'Electric', 7, 'available'],
46     ['B-02', 'B', 'SUV', 6, 'maintenance'],
47     ['C-01', 'C', 'Compact', 4, 'available'],
48     ['C-02', 'C', 'Compact', 4, 'available'],
49     ['D-01', 'D', 'VIP', 10, 'available']
50   ];
51   const tx = db.transaction((rows) => rows.forEach(row => insert.run(...row)));
52   tx(slots);
53 }
54
55 const reservationCount = db.prepare('SELECT COUNT(*) AS count FROM reservations').get().count;
56 if (reservationCount === 0) {
57   const slot = db.prepare('SELECT id FROM slots WHERE code = 'A-03').get();
58   if (slot) {
59     db.prepare(`
60       INSERT INTO reservations (user_name, user_email, vehicle_plate, slot_id, booking_date, start_time, duration_hours, status)
61       VALUES (?, ?, ?, ?, ?, ?, ?, ?)
62     `).run('Alice Johnson', 'alice@example.com', '34 ABC 123', slot.id, '2026-04-16', '09:00', 2, 'active');
63   }
64 }
65
66 module.exports = db;
67
```

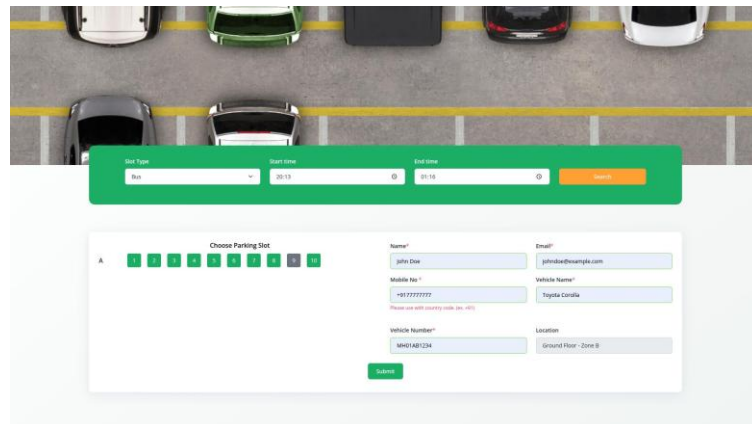
6.3 User interfaces





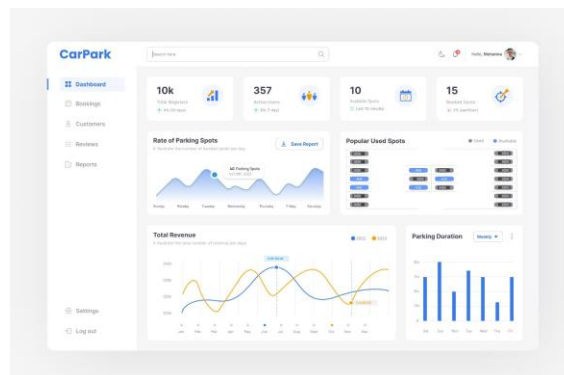
1. Login Page

- ✓ User email input field
- ✓ Password field
- ✓ Login button
- ✓ Register link for new users



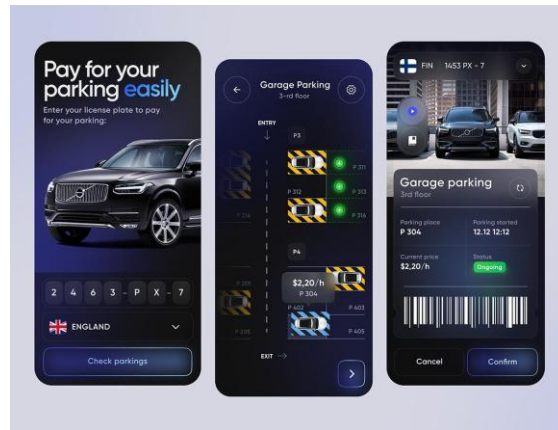
2. User Dashboard

- ✓ Available parking slots display
- ✓ Search/filter options
- ✓ Reservation button
- ✓ Reservation history section



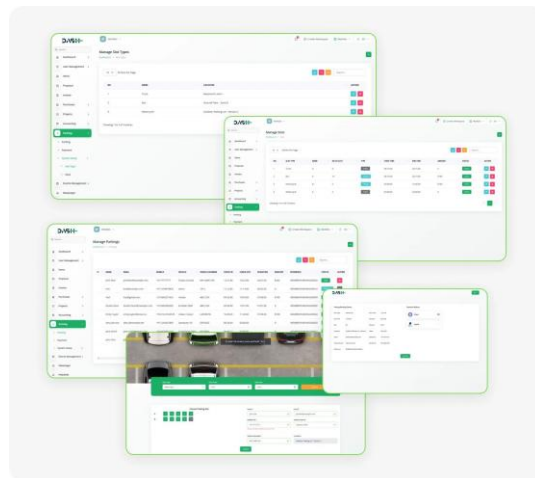
3. Reservation Page

- ✓ Parking slot selection
- ✓ Reservation date/time fields
- ✓ Payment
- ✓ Confirmation button



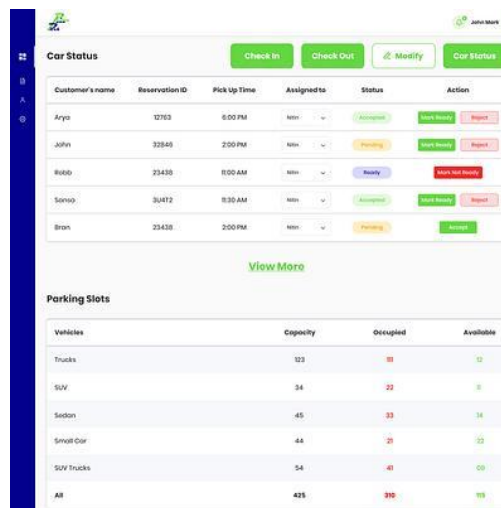
4. Reservation History

- ✓ Current reservations
- ✓ Cancel reservation option
- ✓ Reservation status indicators



5. Admin Dashboard

- ✓ Manage parking slots
- ✓ Add/Edit/Delete slot functions
- ✓ Reservation monitoring
- ✓ Report generation tools



Car Status

Customer's name	Reservation ID	Pick Up Time	Assigned to	Status	Action
Arya	12753	8:00 PM	Nikhil	Assigned	Check Status Report
Ashin	32848	2:00 PM	Nikhil	Pending	Check Status Report
Rubab	23438	10:00 AM	Nikhil	Ready	Check Status Report
Somnath	31472	8:30 AM	Nikhil	Assigned	Check Status Report
Brish	23438	2:00 PM	Nikhil	Pending	Check Status Report

[View More](#)

Parking Slots

Vehicles	Capacity	Occupied	Available
Trucks	123	88	35
SUV	34	22	12
Sedan	45	33	12
Small Car	44	21	23
SUV Trucks	54	40	14
All	425	202	223

6.4 Open-source code

Several open-source technologies and libraries were used during the development of the Parking Slot Reservation System to improve development speed and maintain software quality.

Bootstrap

Bootstrap was used to create responsive and modern user interfaces. Prebuilt components such as forms, buttons, navigation bars, and cards were customized to fit the system requirements.

Node.js / Django

The backend framework handled server-side operations, API development, authentication, and database communication.

MySQL / PostgreSQL

Used for storing user information, parking slots, and reservation records.

Git

Git was used for version control and tracking project changes during development.

Adaptation Process

Open-source components were modified and integrated into the application instead of being copied directly. Custom logic was implemented for:

- Reservation management
- Parking slot availability tracking
- User authentication
- Prevention of double booking

This approach reduced development time while ensuring the system met project requirements.

Chapter 7: Testing

7.1 Unit testing approach

The Parking Slot Reservation System was tested using a unit testing approach to ensure that each component and function of the software works correctly in isolation before integration into the complete system. Each function was tested independently using predefined inputs and expected outputs. The backend API endpoints and frontend user interactions were verified to ensure proper communication with the database.

Tool	Purpose
Jest / Mocha	Backend unit testing
Postman	API endpoint testing
React Testing Library	Frontend component testing
SQLite Testing Database	Temporary database for test execution

The testing process focused on the following modules:

- User Authentication Module
- Parking Slot Management Module
- Reservation Module
- Cancellation Module
- Admin Dashboard Module
- Database Operations Module

Unit Testing Objectives

- Verify correctness of functions
- Detect errors early
- Ensure database consistency
- Prevent double-booking of parking slots
- Validate user inputs and system responses

7.2 Test Cases

Test Case ID	Functionality	Test Description	Input	Expected Result	Status
TC-01	User Registration	Register new user	Valid user details	User account created successfully	Pass
TC-02	User Login	Login with valid credentials	Email & password	User redirected to dashboard	Pass
TC-03	View Parking Slots	Display available slots	User opens dashboard	Available slots displayed	Pass
TC-04	Slot Reservation	Reserve available parking slot	Select slot A1	Reservation successful	Pass
TC-05	Double Booking Prevention	Reserve already reserved slot	Slot A1 already booked	Reservation denied	Pass
TC-06	Reservation Cancellation	Cancel reservation	Reservation ID	Slot becomes available again	Pass
TC-07	Add Parking Slot	Admin adds new slot	Slot information	New slot added to system	Pass
TC-08	Update Slot Status	Admin changes slot status	Maintenance status	Slot marked unavailable	Pass
TC-09	Database Storage	Save reservation to database	Reservation data	Data stored correctly	Pass
TC-10	Search and Filter	Filter parking by zone	Zone B	Only Zone B slots shown	Pass

7.3 Acceptance Testing

User Acceptance Testing (UAT) was conducted with sample users to evaluate whether the system meets the intended requirements and user expectations.

User Requirement	User Feedback	Result
Easy reservation process	Users found booking simple and fast	Accepted
Clear interface design	Users understood navigation easily	Accepted
Real-time slot availability	Slots updated correctly after reservation	Accepted
Admin management tools	Admin users could manage slots effectively	Accepted
Reservation cancellation	Cancellation process worked properly	Accepted
Mobile responsiveness	Interface worked well on smaller screens	Accepted

Chapter 8: Conclusion

8.1 Project Summary

The Parking Slot Reservation System was developed to solve the problem of inefficient parking management and the difficulty users face when searching for available parking spaces manually.

The system provides a web-based platform where users can:

- View available parking slots
- Reserve parking spaces online
- Cancel reservations
- Monitor booking details

Administrators can:

- Add and manage parking slots
- Update slot availability
- Monitor reservations
- Generate parking usage information

Throughout the project, several software engineering concepts were applied, including:

- Requirements analysis
- UML modeling
- Frontend and backend development
- Database integration
- System testing

The final product is a functional full-stack parking reservation system with a modern user interface and working backend/database integration.

The majority of users reported that the system was intuitive, responsive, and effective for managing parking reservations. Minor UI improvements were suggested, such as adding dark mode and payment integration for future development.

8.2 Challenges Faced & Lessons Learned

Challenges Faced

1. Database Synchronization

One of the main technical challenges was ensuring that parking slot availability updates correctly after a reservation or cancellation. Preventing duplicate bookings required careful backend validation.

2. Frontend–Backend Integration

Connecting the React frontend with the backend API required proper handling of asynchronous requests and state updates.

3. User Interface Design

Designing an interface that is both visually appealing and easy to use required multiple revisions and testing iterations.

4. Team Collaboration

Coordinating tasks among team members and merging code changes through GitHub sometimes caused conflicts that needed to be resolved carefully.

5. Time Management

Balancing project deadlines with testing, debugging, and documentation was challenging during the final development stages.

Lessons Learned

1. Importance of proper software planning before implementation
2. Benefits of version control systems like Git and GitHub
3. Practical experience with full-stack development
4. Importance of testing in detecting system errors early
5. Improved teamwork and communication skills
6. Better understanding of database management and API integration

The project also provided hands-on experience in applying software engineering principles to a real-world problem.

References

1. Pressman, R. S. Software Engineering: A Practitioner's Approach. McGraw-Hill Education, 2019.
2. Sommerville, I. Software Engineering. Pearson Education, 2016.
3. React Documentation.
React Official Documentation
4. Node.js Documentation.
Node.js Official Website
5. Express.js Documentation.
Express.js Official Documentation
6. SQLite Documentation.
SQLite Official Website
7. Bootstrap Documentation.
Bootstrap Official Website
8. Chatgpt

Appendices

Appendix A: Sample Screenshots

- User Dashboard
- Reservation Form
- Reservation Management Screen
- Admin Dashboard

Appendix B: Database Schema

Tables

- Users
- ParkingSlots
- Reservations
- Admins

Appendix C: API Endpoints

Method	Endpoint	Description
GET	/slots	Retrieve parking slots
POST	/reserve	Create reservation
DELETE	/reservation/:id	Cancel reservation
POST	/admin/add-slot	Add parking slot
PUT	/admin/update-slot	Update slot status

Appendix D: Future Improvements

- Online payment integration

- QR code parking validation
- Mobile application version
- GPS navigation support
- Email/SMS notifications